

逆向分析完整过程（MCP 动作 × Ghidra GUI 操作并列说明）

下面把整个分析流程重新走一遍。每一步给出两条并列记录：一条是我在 MCP 里实际执行的工具调用，另一条是在 Ghidra GUI（CodeBrowser 窗口）里用鼠标和键盘做同样事情的具体路径。两种做法看到的是同一套数据，选哪条路径只取决于你当前手上是脚本还是 GUI。

下文里凡提到"GUI"，都指 Ghidra 的 CodeBrowser 窗口。Ghidra 里"符号树（Symbol Tree）"默认在左侧，"Listing"（反汇编）在中间，"Decompiler"（反编译）在右侧，"Bookmarks / Console"在下方。

前置操作：打开工程并定位到 CodeBrowser

在深入分析前，假设你已经：

1. 启动 Ghidra，在 Project Window 里 `File → New Project` 建立一个项目。
2. 用 `File → Import File...` 把目标 DLL 导进项目，让 Ghidra 跑完 Auto-Analysis（默认选项勾全就好）。
3. 双击项目里的文件，打开 CodeBrowser。这是后面所有 GUI 操作的入口。

MCP 侧对应的"前置"就是 Ghidra MCP Server 启动后连接到这个打开的 CodeBrowser 会话，不需要显式调用。

第一阶段：整体侦察——这到底是个什么二进制

第 1 步：查看内存段，判定文件类型和基址

MCP 动作

```
ghidra:list_segments()
```

返回了 `.text / .rdata / .data / .pdata / .fptable / .rsrc / .reloc / tdb` 的布局，镜像基址 `0x180000000`。

GUI 操作

- 菜单栏点 `Window → Memory Map`，弹出的 Memory Map 窗口里会看到完全一样的段表。
- 或者在顶部工具栏找到那个看起来像小芯片/小砖头堆叠的图标（Memory Map 快捷按钮）。

读到了什么 镜像基址 `0x18000_0000` 是 Windows x64 DLL 的默认首选加载地址，`0x400000` 才是 EXE。段组合里还有 `.pdata`（x64 SEH 展开表）说明是 MSVC 编译的 PE。这一步就得到"64 位 DLL，MSVC 编译"这个大前提。

第2步：搜关键词 **AOB**，摸清作者的命名体系

用户的问题里已经出现 "AOB"，这是 Cheat Engine / trainer 圈里的专用词，先用它当钩子能省非常多时间。

MCP 动作

```
ghidra:list_strings(filter="AOB", limit=200)
```

返回了几十条字符串，包括：

- **AOB_BL4_COORDS_WRITE_LOCATION**、**AOB_SHF_FOV_WRITE_LOCATION** 等 key 名
- 错误消息 "The action of AOB %s couldn't be enabled as it's not set!"
- RTTI 类名 **?.?AVAObHookAction@Hooking@IGCS@@**、**?.?AVAObNopRangeAction@Hooking@IGCS@@**、**?.?AVAObWriteRangeAction@Hooking@IGCS@@**

GUI 操作

- 菜单 **Window → Defined Strings**（或顶栏工具条里双引号图标 ）打开 Defined Strings 窗口。
- 窗口下方有 **Filter** 输入框，输入 **AOB** 回车即可。
- 也可以走 **Search → For Strings...** 先重新扫一遍整个程序的字符串（Ghidra 导入时默认只对 **.rdata** 做了字符串识别，如果漏就用这条补）。

读到了什么 三件事一次性锁定：

1. 命名空间是 **IGCS::Hooking**，这是 Otis_Inf 公开的 **Injectable Generic Camera System** 框架。
2. 作者采用**命名签名表**架构：所有 AOB 都用字符串 key 在一个中心表里注册 / 查询，不是四处散落的裸字节。这种设计对逆向者极友好，找到"注册点"就等于一次性扒出整张表。
3. key 的前缀暗示游戏，**AOB_BL4_***（Borderlands 4）、**AOB_SHF_***（Silent Hill f），作者按游戏分组。

第二阶段：定位 AOB 注册函数

第3步：从一条独特字符串反查交叉引用，找到第一个登记点

选一条比较独特、不会被无关代码引用的 key 名，比如

"AOB_BL4_MAINTHREAD_CALL_INTERCEPT"（在第2步的字符串列表里能看到它的地址 **0x180272738**）。

MCP 动作

```
ghidra:get_xrefs_to(address="0x180272738")
```

返回两个引用点：`FUN_1800fe350` 和 `FUN_1800fe500`。

GUI 操作

- 在 Listing 窗口任何位置按 `G` (Go To)，输入 `180272738`，回车，光标跳到该字符串所在行。
- 字符串行左边会有一个像 `s_AOB_BL4_MAINTHREAD_CALL_INTERCEPT_180272738` 的标签。右键这个标签，弹出菜单里选 `References → Show References to Address`。
- 弹出的 References 表里就会列出 `1800fe42d` (属于 `FUN_1800fe350`) 和 `1800fe58d` (属于 `FUN_1800fe500`) 两行。双击任一行即可跳过去。
- 快捷键也行：光标停在标签上按 `Ctrl+Shift+F`，直接打开 References 表。

第 4 步：反编译第一个登记函数，认出"注册模板"

MCP 动作

```
ghidra:decompile_function_by_address(address="0x1800fe350")
```

屏幕上出现一段高度模板化的代码：

```
c
puVar1 = (undefined8 *)FUN_1801d8974(0x418);
puVar1 = FUN_18000b790(puVar1, 0xc);
puVar2 = FUN_1800171a0(local_60,
    "48 89 46 10 0F 28 44 24 20 0F 11 06 48 8B 4C 24 40 48 31 E1 48 3B 0D");
puVar3 = FUN_1800171a0(local_40, "AOB_BL4_COORDS_WRITE_LOCATION");
FUN_180108570((longlong)param_1, puVar3, puVar2, 1, puVar1);
```

GUI 操作

- 沿用第 3 步的路径：双击 References 表里 `1800fe42d` 那一行，光标自动跳进 `FUN_1800fe350`。
- 右侧的 **Decompiler** 窗口会同步显示这个函数的 C 伪代码。Decompiler 默认是打开的；如果不见了，用 `Window → Decompiler` 调出来。
- 在 Decompiler 里想跳到调用的子函数，按住 `Ctrl` 鼠标左键点函数名即可跟进去；返回用 `Alt+左箭头`。

读到了什么 四行一组的"登记模板"非常显眼：

这一行做的事	对应函数含义
<code>operator new(0x418)</code>	<code>FUN_1801d8974</code> = 内存分配器
调用构造器填 <code>vtable</code> 和字段	<code>FUN_18000b790</code> / <code>_b590</code> / <code>_b8a0</code> 对应 <code>AOBHookAction</code> / <code>AOBNopRangeAction</code> / <code>AOBWriteRangeAction</code> 三种 Action 子类构造函数
构造 <code>std::string("字节模式")</code>	<code>FUN_1800171a0</code> = 字符串字面量构造器
构造 <code>std::string("AOBkey 名")</code>	同上
把 key 名、模式、Action 登记到某个表里	<code>FUN_180108570</code> = 注册函数本体

后面只要再看到这种"四五行一组"的格式，就可以直接判定是一条 AOB 登记。

第 5 步：反向展开注册函数，枚举全部登记站点

MCP 动作

```
ghidra:get_xrefs_to(address="0x180108570", limit=200)
```

返回 30 多个调用点，去重后归属到九个注册入口函数：`FUN_1800b2de0`、`FUN_1800eb060`、`FUN_1800fe350`、`FUN_1800fec80`、`FUN_1800fef50`、`FUN_1800ff0b0`、`FUN_180101210`、`FUN_1801155d0`、`FUN_180138800`。

GUI 操作

- 在 Listing 里按 `G`，跳转到 `180108570`（或者在左侧 Symbol Tree 展开 `Functions` 节点，找到 `FUN_180108570` 双击）。
- 函数入口行上右键 → `References → Show References to Function`。弹出的表会列出所有调用点。
- 另一条更直观的路线是 `Window → Function Call Trees` 打开调用树窗口，把 `FUN_180108570` 拖进去，左边面板 "Incoming Calls" 就是全部调用者。
- 想顺手把这些函数改名方便以后找，在 Listing 的函数名上按 `L`（label/rename）输入新名字即可。比如我可以把 `FUN_180108570` 改成 `IGCS_Hooking_RegisterAOBAction`，对应 MCP 动作是 `ghidra:rename_function_by_address`。

读到了什么 这九个函数就是整个 DLL 里所有 AOB 登记入口的全集。剩下的工作只是把它们一个个反编译、把每条 `key × 字节串` 抄下来。

第三阶段：区分"通用 UE"与"按游戏分支"的登记函数

我把九个函数逐个反编译（方法和第 4 步完全一样，只是换地址）。过程中发现它们按内部结构分两类，这个分类对理解整张 AOB 表非常关键。

第 6 步：通用 UE 函数——同一个 key 登记多个"候选字符串"

以 `FUN_1800b2de0` 为例，反编译输出里能看到对同一个 key

`"AOB_FILLCOMPONENTSPACETRANSFORMS_CALL"` 被注册了 26 次，每次带一串略有差异的字节模式：

```
c

// 第 1 个变体
FUN_1800171a0(local_80, "4C 8D 86 ?? ?? 00 00 4D 8B CC 48 8B D7 49 8B CE E8 | ?? ??");
FUN_1800171a0(local_a0, "AOB_FILLCOMPONENTSPACETRANSFORMS_CALL");
FUN_1801084c0(...); // 第一个用"SET" (1801084c0)，之后用"ADD" (180108700)

// 第 2 个变体
FUN_1800171a0(local_80, "4C 8B 4D 5F 4D 8D 87 ?? ?? 00 00 48 8B D3 49 8B CD E8 | ??");
FUN_1800171a0(local_a0, "AOB_FILLCOMPONENTSPACETRANSFORMS_CALL");
FUN_180108700(...);
... (以此类推到第 26 个)
```

接着又对 `"AOB_REQUIREDBONESOFFSET_LOCATION"` 重复同样的 26 条。

MCP 动作：反复 `ghidra:decompile_function_by_address` 每个入口函数。

GUI 操作：

- Listing 里按 `G` 跳到入口地址；或者在 Decompiler 里按住 `Ctrl` 点函数名跟进。
- 在 Decompiler 里用 `Ctrl+F` 可以对当前反编译结果做关键字搜索，比如输 `AOB_FILL`，会高亮所有出现的行，手动数或抄下来。
- Decompiler 窗口顶部右侧有一个"复制"按钮（或按 `Ctrl+A` → `Ctrl+C`），可以把整个函数的伪代码拷出来到本地编辑器再用正则抓模式串。
- 想更直观观察，可以右键 Decompiler → `Display Function Call Graph`（图形化调用图），看哪些函数是"登记工厂"。

读到了什么 同一个 key 登记这么多条的原因是：UE 引擎在不同游戏里被不同版本编译器 / 不同优化等级编译，同一个逻辑位置产生的字节会漂移。作者做了"全 UE 版本候选签名"，匹配到哪一条就用哪一条。这解释了为什么 AOB 表规模很大。

第7步：按游戏分支函数——内部带 `if(processName == "...")` 分发

反编译 `FUN_180101210`（camera struct / black bars 登记函数）和 `FUN_1800eb060`（写大气雾登记函数）时，结构完全不同。以 `FUN_1800eb060` 为例：

```
c
cVar1 = FUN_1800cd250(local_58, "TheOuterWorlds2-Win");
if (cVar1 != '\0') {
    puVar4 = FUN_1800171a0(local_78,
        "E8 ?? ?? ?? ?? 80 BB 50 0A 00 00 00 74 0D 48 8B CB 48 83 C4 20");
    puVar5 = FUN_1800171a0(local_98, "AOB_CUSTOM_WRITEATMOSPHERICS_CALL_LOCATION");
    FUN_180108570(..., 1, puVar3);
}
cVar1 = FUN_1800cd250(local_58, "Avowed-Win");
if (cVar1 != '\0') { ...另一条字节模式... }
cVar1 = FUN_1800cd250(local_58, "OblivionRemastered-Win");
if (cVar1 != '\0') { ...又一条... 甚至两条... }
cVar1 = FUN_1800cd250(local_58, "CodeVein2-Win");
if (cVar1 != '\0') { ...又一条... }
```

这是非常标准的运行时分发：读出进程的可执行名（`FUN_1801125e0 → FUN_1800b25b0`）这条链，`FUN_1800cd250 / FUN_1800cd290` 是 `std::string::compare` 的两种重载），然后用字符串比对挑分支。

MCP 动作：同样是 `ghidra:decompile_function_by_address`。

GUI 操作：

- Decompiler 里把光标停在 `FUN_1800cd250` 这种频繁出现的辅助函数名上，`Ctrl+左键` 跟进一次，看清它就是 `std::string::compare` 的返回值（非零表示匹配）。确认一次之后可以按 `(L)` 把它改名为 `string_compare_returns_match`，MCP 对应 `ghidra:rename_function_by_address`，改名后所有反编译 C 代码里都会同步显示新名字，可读性暴增。
- 再把 `FUN_1801125e0 / FUN_1800b25b0` 这条链也改名成 `GetCurrentProcessExecutableName`，整个分支结构就变得像手写代码一样好读。

读到了什么 两类函数确认：通用 UE 注册函数负责给所有 UE 游戏登记“万能候选签名”；按游戏分支函数在进程名匹配特定游戏时，给某些 key 追加/替换该游戏专属的字节模式。两类叠在一起构成了最终运行时的 AOB 表。

第四阶段：把进程代号翻译成真实游戏名

抄下来的进程名有几个是看不出来的内部代号——`Polaris-Win64-Shipping`、`SHf-Win64-Shipping`、`M1-Win64-Shipping`、`b1-Win64-Shipping`、`TQ2-Win64-Shipping`。要想翻译，必

须换一个入口点进攻。

第 8 步：先枚举所有"看起来像进程名"的字符串

MCP 动作

```
ghidra:list_strings(filter="-Win", limit=200)
```

过滤出包含 `-Win` 的字符串，能一次性列出所有注入目标：`TheOuterWorlds2-Win`、`Avowed-Win`、`OblivionRemastered-Win`、`CodeVein2-Win`、`Banishers-Win64-Shipping`、`M1-Win64-Shipping`、`Stalker2-Win`、`Hellblade2-Win64-Shipping`、`SHf-Win64-Shipping`、`Polaris-Win64-Shipping`、`TQ2-Win64-Shipping`、`b1-Win64-Shipping`.....

GUI 操作

- `Window → Defined Strings`，在 `Filter` 框输入 `-Win`。
- 或 `Search → For Strings...` 弹出对话框，勾选 `Search Memory Blocks`，点 `Search`，结果窗口里同样可以 `filter`。
- 每条结果右键 → `Go To` 跳到字符串所在地址；想看它谁在引用就接第 3 步的 `References → Show References to Address`。

第 9 步：找"游戏分发主入口"，读类名拿真名

四款游戏在 `DLL` 里有专属 `Feature` 类。分发主入口会根据进程名选择实例化哪个类。最快找到它的方法是对 `"Hellblade2-Win64-Shipping"` 这条字符串做交叉引用（因为 `Hellblade2` 只被主分发入口引用，干扰少）。

MCP 动作

```
ghidra:get_xrefs_to(address="0x1802759e8") # Hellblade2-Win64-Shipping 的地址
```

返回 `FUN_18010ca10` 一个引用点。

```
ghidra:decompile_function_by_address(address="0x18010ca10")
```

伪代码里看到：

```
c
```

```

cVar1 = thunk_FUN_1800cd290(local_108, "Polaris-Win64-Shipping");
if (cVar1 != '\0') {
    plVar8 = FUN_1800ff070(puVar2);    // 分配并初始化 Tekken8Feature
}
cVar1 = thunk_FUN_1800cd290(local_108, "Hellblade2-Win64-Shipping");
if (cVar1 != '\0') {
    plVar6 = FUN_1800febf0(puVar2);    // Hellblade2Feature
}
cVar1 = thunk_FUN_1800cd290(local_108, "Borderlands4");
if (cVar1 != '\0') {
    plVar6 = FUN_1800fe240(puVar2);    // Borderlands4Feature
}
cVar1 = thunk_FUN_1800cd290(puVar2, "SHf-Win64-Shipping");
if (cVar1 != '\0') {
    pcVar11 = (char *)FUN_1800feed0(puVar2);    // SilentHillfFeature
}

```

GUI 操作

- 按 **G** 跳到 **1802759e8**，看到 **"Hellblade2-Win64-Shipping"** 行。
- 右键行首的标签 → **References → Show References to Address**，弹出 **References** 表，只有一条记录 (**18010cc91**)。
- 双击这一条，进入 **FUN_18010ca10**，Decompiler 自动显示伪代码。
- Decompiler 里 **Ctrl+F** 搜 **"-Shipping"** 能高亮所有进程名比对。

第 10 步：逐个跟进"派生类构造器"，读它们自报家门的描述字符串

MCP 动作

```

ghidra:decompile_function_by_address(address="0x1800ff070")    # Polaris 分支
ghidra:decompile_function_by_address(address="0x1800febf0")    # Hellblade2 分支
ghidra:decompile_function_by_address(address="0x1800fe240")    # Borderlands4 分支
ghidra:decompile_function_by_address(address="0x1800feed0")    # SHf 分支

```

每个函数都只有十几行，都是 C++ 派生类构造器的教科书写法：

```

c
puVar1 = FUN_1800171a0(local_28, "Tekken 8 specific features");
FUN_180108240(param_1, puVar1);
*param_1 = IGCS::GameSpecific::FeaturesPerGame::Tekken8Feature::vftable;

```

第一行的描述字符串就是代号到真名的翻译。

GUI 操作

- 在 `FUN_18010ca10` 的 Decompiler 里，按 `Ctrl+左键` 分别点每个 `FUN_1800ff070`、`FUN_1800febf0`、`FUN_1800fe240`、`FUN_1800feed0`，跟进去直接看。
- 更关键的是看"vtable 指针赋值"这一行。Ghidra 会把 vtable 显示成 `PTR_FUN_XXXXX` 之类的数据标签。按 `G` 跳到那个地址，Listing 里会看到 Ghidra 已经自动识别出 RTTI，标签名里就包含 `Tekken8Feature@FeaturesPerGame@GameSpecific@IGCS`，直接给你把类名吐出来。
- 另一种纯 GUI 的路线：`Window → Classes`（Symbol Tree 里也有 Classes 节点），Ghidra 用 MSVC RTTI 解析出的所有 C++ 类都在这里，展开就能看到 `IGCS::GameSpecific::FeaturesPerGame::Tekken8Feature` 等节点。展开类节点还能看到它的 vtable 和每个虚函数。

读到了什么 四个代号一次性翻译完：

代号	真名
<code>Polaris-Win64-Shipping</code>	Tekken 8（Polaris 是 Bandai Namco 对 Tekken 8 的内部代号）
<code>SHf-Win64-Shipping</code>	Silent Hill f
<code>Hellblade2-Win64-Shipping</code>	Senua's Saga: Hellblade II
<code>Borderlands4</code>	Borderlands 4

第 11 步：核实是不是只有这四款游戏有专属类

MCP 动作

```
ghidra:list_strings(filter="Feature", limit=200)
```

结果里 `FeaturesPerGame` 命名空间下只出现四个派生类：`Borderlands4Feature`、`Hellblade2Feature`、`SilentHillfFeature`、`Tekken8Feature`。其他游戏名都不在这个命名空间里。

GUI 操作

- Defined Strings 窗口 Filter 输入 `FeaturesPerGame`，一眼数完。
- 或展开 `Window → Classes` 的 Class 树，从 `IGCS / GameSpecific / FeaturesPerGame` 节点往下看，只有四个孩子。

结论

- Borderlands 4 / Tekken 8 / Hellblade 2 / Silent Hill f 有完整的专属 Feature 类，说明适配最深。
- 其余游戏（Avowed、Stalker 2、Oblivion Remastered、Banishers、TheOuterWorlds2、CodeVein2、M1、b1、TQ2）都是共用通用 UE 代码路径，只在 AOB 表里打少量"补丁"条目。
- 因此 (M1) / (b1) / (TQ2) 这三个代号在本 DLL 内部**没有**任何辅助线索（没有类名、没有自报家门字符串、没有日志），只能靠外部知识或运行时抓进程名来翻译。不过可以留下两条辅助推断：
 - (b1) 触发的是 UE 4.27 的旧姿势编辑器路径（在 (FUN_1800b3fb0) 里 (b1-Win64-Shipping) 匹配后会走 (AOB_FILLCOMPONENTSPACETRANSFORMS_LOCATION_427) 这条带 (427) 后缀的 key），说明它是 UE 4.27 编译的。
 - (M1) 和 (Stalker 2) 共用 (AOB_BLOCK_GAMEPAD_INPUT_LOCATION_KEY)，说明 UE5 且有类似的手柄输入拦截需求。

第五阶段：整合与复核

第 12 步：整理最终的 AOB × 游戏映射表

方法就是把第 5 步找到的九个注册函数全部反编译结果，按下面的规则分类合并：

- 出现在 (if (processName == X)) 分支里的字节模式 → 归属 X 游戏的**专属 AOB**。
- 没有任何 if 保护直接登记的 → **通用 UE AOB**（统计变体数量）。
- 出现在 (FeatureXxx::init) 里的额外登记（比如 BL4 专属那三条）→ 也算游戏专属。

最终合并表前面两轮回答里已经给过，这里只强调一点：**游戏专属 AOB 大概只占整张表的 10% 不到，其余 90% 都是通用 UE 候选签名**。这是 IGCS 框架的核心设计——它假设 UE 游戏大同小异，只在无法容忍的地方做游戏专属覆盖。

GUI 操作（可选的增强可读性步骤） 做到这里时，一个很实用的 GUI 动作是**批量给未命名函数改名**：

- 在 Symbol Tree 的 Functions 节点按 (F2) 或右键 Rename，把 (FUN_180108570) 改成 (IGCS_Hooking_RegisterAOBAction)；
- (FUN_1800171a0) 改成 (std_string_ctor)；
- (FUN_1800cd250) / (FUN_1800cd290) 改成 (std_string_contains) / (std_string_compare)；
- (FUN_1801125e0) 改成 (IGCS_GetCurrentProcess)；
- (FUN_1800b25b0) 改成 (IGCS_GetExecutableName)；
- (FUN_1801d8974) 改成 (operator_new)。

这些改名动作的 MCP 等价是 (ghidra:rename_function_by_address)。改完之后，再回去反编译任何一个注册函数，伪代码的可读性会接近手写 C++。同样的操作 GUI 侧就是：Listing 里光标停在函数名上按 (L) 或 (F2)，在弹出的对话框里输入新名。

第13步：核对没有遗漏

最后再扫一遍整张字符串表，看有没有漏网的 AOB key：

MCP 动作

```
ghidra:list_strings(filter="AOB_", limit=500)
```

把返回的每个 key 和前面整理的表对一遍。有几个 key (`AOB_NAMESSTORE`、`AOB_OBJECTSSTORE`、`AOB_ENGINEVERSION`、`AOB_GENGINE`、`AOB_UOBJECT_PROCESSEVENT` 等) 只出现在字符串表和某些查询调用里，没有出现在我找到的九个注册函数里——这意味着它们是在另外的模块里登记的（很可能在 Unreal 模块初始化代码 `FUN_180138800` 之外的 SDK 扫描函数里），如果严格追求"穷尽所有 AOB"，下一步就是针对这些 key 再各自做一次第 3 步的交叉引用回溯。

GUI 操作 Defined Strings 窗口 filter `AOB_`，按地址排序后肉眼扫一遍有没有生面孔。对每个陌生 key 重复"右键 → References → Show References to Address"即可。

整个方法论的抽象总结

把上面 13 步抽象出来，这类"分析一个插件式 DLL 的特征表"的通用流程是这样的：

1. 先看段表判文件类型 (MCP: `list_segments` ; GUI: `Window → Memory Map`)。
2. 搜关键词字符串捡钩子 (MCP: `list_strings(filter=...)` ; GUI: `Window → Defined Strings` + Filter)。这是成本最低、回报最高的动作，应该先做。
3. 选一条独特字符串做 xref to，找到一处"注册调用点" (MCP: `get_xrefs_to` ; GUI: 字符串标签上右键 → `References → Show References to Address`，或 `Ctrl+Shift+F`)。
4. 反编译调用点，抽象出"注册模板"的四五行结构 (MCP: `decompile_function_by_address` ; GUI: 按 `G` 跳转 + 看右侧 Decompiler, `Ctrl+左键` 跟进子函数)。
5. 对注册函数做 xref to，一次性枚举全部登记入口 (MCP: `get_xrefs_to` on 注册函数地址 ; GUI: 函数入口右键 `References → Show References to Function`，或 `Window → Function Call Trees`)。
6. 按入口函数逐个反编译，区分"通用多候选"和"按进程名分支"两类。
7. 找游戏分发主入口 (在本例里是 `FUN_18010ca10`)：选一条字符串 (例如 `Hellblade2-Win64-Shipping`) 做 xref to，几乎必然命中主入口。
8. 通过派生类构造器的自报家门字符串，把内部代号翻译成真实游戏名 (GUI: 也可以走 `Window → Classes` 看 RTTI 推出的类树)。
9. 交叉合并两边结果，得到 `AOB key × 游戏 × 字节模式` 的完整表。
10. 给核心函数 / 数据批量改名 (MCP: `rename_function_by_address` ; GUI: `L` 或 `F2`)，让最终的伪代码接近手写，便于写文档或分享。
11. 用字符串表回扫一遍漏网 key，对陌生 key 再做一次 xref to 回溯，直到穷尽。

整个过程里没有修改任何字节、没有 **patch** 跳转，是**纯静态、纯读取**的分析——本质上我们做的事情只是"用 **Ghidra** 把一个设计良好的开源工具内部的数据结构读了出来"。**MCP** 工具和 **GUI** 操作完全等价，区别只在于前者更适合批处理和聊天里复现，后者更适合人坐在屏幕前慢慢探索。两者可以自由切换。